NM-ROM math explainer

What the code is actually doing, line by line

(written for Tahmid)

What this document is

Working through every piece of math in your NM-ROM solver, from scratch. Two PDEs (Poisson and Heat), two solvers, both Galerkin. Nothing assumed beyond linear algebra and what a derivative is. We will keep the notation simple: bold lower-case for vectors, bold upper-case for matrices, plain italic for scalars.

The whole document follows your code, not textbooks. Every equation here maps to a specific line in your repository.

Contents

---

1 The setup: what N and k mean

We discretize a PDE on a 3D mesh with N_g points along each axis. Total number of unknowns:

$$N \;=\; N_g^d, \qquad d \in \{2, 3\}.$$

For $N_g = 64$, $d = 3$ that is $N = 262{,}144$. The full PDE state is a vector $\mathbf{u} \in \mathbb{R}^N$ giving the solution value at each mesh point.

We want to replace this N-dimensional thing with a k-dimensional *latent code* $\mathbf{z} \in \mathbb{R}^k$. Typical k in your experiments: 8, 16, 24. So we are compressing a 262,144-dim thing into a 24-dim thing. That is a 10,000× reduction.

The compression is done by an autoencoder:

$$\text{encoder } \mathcal{E} : \mathbb{R}^N \to \mathbb{R}^k, \qquad \text{decoder } \mathcal{D} : \mathbb{R}^k \to \mathbb{R}^N.$$

After training, $\mathcal{D}(\mathcal{E}(\mathbf{u})) \approx \mathbf{u}$ for $\mathbf{u}$ that look like training snapshots.

The masked decoder. Your code does not use $\mathcal{D}$ directly — it wraps it with a Dirichlet mask:

$$\tilde{\mathbf{u}}(\mathbf{z}) \;=\; \mathbf{m} \odot \mathcal{D}(\mathbf{z}) \;+\; \mathbf{u}_g.$$

Here $\mathbf{m} \in \{0, 1\}^N$ is a binary mask that is 0 at boundary nodes and 1 at interior nodes, $\mathbf{u}_g$ is the prescribed boundary lift (the known boundary values), and $\odot$ is element-wise multiplication. So $\tilde{\mathbf{u}}$ is forced to take the right boundary values regardless of what the network outputs at boundary nodes.

This $\tilde{\mathbf{u}}(\mathbf{z})$ is what we will plug into the PDE. From now on whenever you see $\tilde{\mathbf{u}}(\mathbf{z})$, think "the masked decoder."

2 The Jacobian J: what it is and why we need it

This is the most important object in the whole paper. Get this and the rest is easy.

Define

$$\mathbf{J}(\mathbf{z}) \;=\; \frac{\partial \tilde{\mathbf{u}}}{\partial \mathbf{z}} \;\in\; \mathbb{R}^{N \times k}.$$

This is an $N \times k$ matrix. Row i, column j is the partial derivative of $\tilde{u}_i$ (the value at mesh node i) with respect to z_j (the j-th latent coordinate):

$$\mathbf{J}_{ij} \;=\; \frac{\partial \tilde{u}_i}{\partial z_j}.$$

What does J mean intuitively? Pick a latent point $\mathbf{z}$. Now wiggle one coordinate, say z_5, by a tiny amount δ. The full-field solution changes by $\delta \cdot \mathbf{J}_{:,5}$, the 5th column of $\mathbf{J}$. So each column of $\mathbf{J}$ is a "direction in $\mathbb{R}^N$ that the manifold can move when you perturb one latent coordinate."

The k columns of $\mathbf{J}$ span the *tangent space* of the non-linear manifold at $\mathbf{z}$. If the manifold were linear (POD), $\mathbf{J}$ would be the same matrix everywhere. Because it is non-linear, $\mathbf{J}$ depends on $\mathbf{z}$.

We never actually form J. At $N = 262{,}144$ and $k = 24$, $\mathbf{J}$ is a $262{,}144 \times 24$ matrix — 6.3 million entries, 25 MB. We could form it, but we do not need to. JAX's autodiff gives us two operators that are equivalent to $\mathbf{J}$ but never store it:

- `jvp` (forward-mode) computes $\mathbf{J}(\mathbf{z})\,\delta\mathbf{z}$ for any $\delta\mathbf{z} \in \mathbb{R}^k$. Output is in $\mathbb{R}^N$.
- `vjp` (reverse-mode) computes $\mathbf{J}(\mathbf{z})^\top \mathbf{v}$ for any $\mathbf{v} \in \mathbb{R}^N$. Output is in $\mathbb{R}^k$.

That is enough to do everything we need.

3 Poisson: the static problem

3.1 What we are solving (no ROM yet)

The Poisson equation with Dirichlet boundary conditions, after FEM discretization, becomes:

$$\mathbf{K}\,\mathbf{u} \;=\; \mathbf{F}, \qquad \mathbf{u}, \mathbf{F} \in \mathbb{R}^N, \qquad \mathbf{K} \in \mathbb{R}^{N \times N}.$$

$\mathbf{K}$ is the stiffness matrix — it encodes the discrete Laplacian. It is symmetric positive definite (SPD) and sparse. We can solve this directly with conjugate gradient (CG) on the full N-dim system. That is the FOM (full-order model) baseline.

The *residual* is the vector

$$\mathbf{R}(\mathbf{u}) \;=\; \mathbf{K}\mathbf{u} - \mathbf{F}.$$

Solving Poisson means driving $\mathbf{R}$ to zero.

3.2 Plug in the masked decoder

Now we replace $\mathbf{u}$ with $\tilde{\mathbf{u}}(\mathbf{z})$. The residual becomes a function of $\mathbf{z}$:

$$\mathbf{R}(\mathbf{z}) \;=\; \mathbf{K}\,\tilde{\mathbf{u}}(\mathbf{z}) - \mathbf{F} \;\in\; \mathbb{R}^N.$$

But wait — this has N equations and only k unknowns. We are over-determined: 262,144 equations, 24 unknowns. We cannot generally make $\mathbf{R}(\mathbf{z}) = \mathbf{0}$ exactly. We need to project the equations down to k of them.

3.3 Galerkin projection

This is where Galerkin comes in. We pick a k-dimensional "test space" to project the residual onto, and require the projected residual to be zero. The natural choice (and the one your code makes) is to use the columns of $\mathbf{J}(\mathbf{z})$ as the test vectors:

$$\mathbf{r}_{\text{red}}(\mathbf{z}) \;=\; \mathbf{J}(\mathbf{z})^\top\,\mathbf{R}(\mathbf{z}) \;=\; \mathbf{J}(\mathbf{z})^\top\big(\mathbf{K}\,\tilde{\mathbf{u}}(\mathbf{z}) - \mathbf{F}\big) \;=\; \mathbf{0}.$$

This is the *reduced residual* $\mathbf{r}_{\text{red}} \in \mathbb{R}^k$. Now we have k equations in k unknowns.

Why test against J? Geometrically: we are saying "the leftover error $\mathbf{R}(\mathbf{z})$ should be orthogonal to the tangent space of the manifold." That is the definition of an optimal projection in the Euclidean norm. This is the classical Galerkin choice.

Galerkin vs LSPG. There is another popular choice: LSPG (Least-Squares Petrov-Galerkin). LSPG uses $\mathbf{KJ}$ as the test space, not $\mathbf{J}$:

$$(\mathbf{KJ})^\top\,\mathbf{R}(\mathbf{z}) \;=\; \mathbf{J}^\top\mathbf{K}^\top(\mathbf{K}\tilde{\mathbf{u}} - \mathbf{F}) \;=\; \mathbf{0}.$$

This is the same thing as minimizing $\|\mathbf{K}\tilde{\mathbf{u}} - \mathbf{F}\|_2$ over the manifold. Your code does *not* do LSPG — there is no $\mathbf{K}^\top$ in your reduced residual. You are doing Galerkin. For SPD $\mathbf{K}$ this is also the energy-minimizing projection, which is why it is the right choice for Poisson and Heat.

3.4 Solving the reduced system: Gauss-Newton

$\mathbf{r}_{\text{red}}(\mathbf{z})$ is a non-linear function of $\mathbf{z}$ because $\tilde{\mathbf{u}}(\mathbf{z})$ is non-linear. So we cannot just invert. We use Newton-style root-finding.

Newton's method for $\mathbf{r}_{\text{red}}(\mathbf{z}) = \mathbf{0}$ says: linearize around the current $\mathbf{z}$, solve the linear system, update.

$$\mathbf{r}_{\text{red}}(\mathbf{z} + \delta\mathbf{z}) \;\approx\; \mathbf{r}_{\text{red}}(\mathbf{z}) + \mathbf{H}(\mathbf{z})\,\delta\mathbf{z},$$

where $\mathbf{H}(\mathbf{z}) \in \mathbb{R}^{k\times k}$ is the Jacobian of $\mathbf{r}_{\text{red}}$ with respect to $\mathbf{z}$. Setting the linearization to zero:

$$\mathbf{H}(\mathbf{z})\,\delta\mathbf{z} \;=\; -\mathbf{r}_{\text{red}}(\mathbf{z}).$$

We need to compute $\mathbf{H}$. By the chain rule on $\mathbf{r}_{\text{red}} = \mathbf{J}^\top(\mathbf{K}\tilde{\mathbf{u}} - \mathbf{F})$:

$$\mathbf{H}(\mathbf{z}) \;=\; \frac{\partial \mathbf{r}_{\text{red}}}{\partial \mathbf{z}} \;=\; \underbrace{\mathbf{J}^\top\mathbf{K}\,\mathbf{J}}_{\text{Gauss-Newton part}} \;+\; \underbrace{\frac{\partial \mathbf{J}^\top}{\partial \mathbf{z}}(\mathbf{K}\tilde{\mathbf{u}} - \mathbf{F})}_{\text{second-order, dropped}}\;.$$

The second term involves second derivatives of $\tilde{\mathbf{u}}$ and is expensive. Gauss-Newton drops it. The approximation is good when the residual $\mathbf{R}$ is small, which is exactly what we want as we converge. So the Gauss-Newton update is:

$$\boxed{\left(\mathbf{J}^\top\mathbf{K}\mathbf{J}\right)\delta\mathbf{z} \;=\; -\mathbf{J}^\top(\mathbf{K}\tilde{\mathbf{u}} - \mathbf{F})}$$

This is a $k \times k$ symmetric positive definite linear system. We solve it with CG (the inner solver). Then $\mathbf{z} \leftarrow \mathbf{z} + \delta\mathbf{z}$. Repeat until $\|\mathbf{r}_{\text{red}}\|$ is small enough.

3.5 Mapping to the code, line by line

This is what `poisson_vitsep_linearcp.py` lines 418–434 do. Pseudocode:

```
def body_fn(state):
    z, _, i = state
    u_pred, vjp_fn = jax.vjp(D, z)        # u_pred = D(z),
                                          # vjp_fn(v) = J(z).T @ v
    R     = K @ u_pred - F                # residual in R^N
    r_red = vjp_fn(R)[0]                  # J.T @ R, in R^k

    def gn_op(dz):                        # this computes
        _, J_dz = jax.jvp(D, (z,), (dz,)) #   J @ dz, in R^N
        K_J_dz  = K @ J_dz                #   K @ J @ dz, in R^N
        return vjp_fn(K_J_dz)[0]          #   J.T @ K @ J @ dz, in R^k

    delta_z, _ = cg(gn_op, -r_red, tol=1e-3)   # solve k x k system
    z = z + delta_z
    return z, ||r_red||, i+1
```

That is exactly the boxed equation. `r_red` is $\mathbf{J}^\top\mathbf{R}$. `gn_op(dz)` computes $\mathbf{J}^\top\mathbf{K}\mathbf{J}\,\delta\mathbf{z}$ matrix-free using one `jvp` (gives $\mathbf{J}\,\delta\mathbf{z}$), one sparse $\mathbf{K}$-apply, and one `vjp` (gives $\mathbf{J}^\top\cdot$). CG only ever sees this operator; it never needs the matrix.

The cold start. The outer loop initializes $\mathbf{z}_0 = \mathbf{0}$. So the first Gauss-Newton step solves

$$\left(\mathbf{J}(0)^\top\mathbf{K}\mathbf{J}(0)\right)\delta\mathbf{z} \;=\; -\mathbf{J}(0)^\top(\mathbf{K}\,\tilde{\mathbf{u}}(0) - \mathbf{F}).$$

For this to be a sensible step, $\mathbf{J}(0)$ has to be well-conditioned. That is the architectural constraint that motivates the linear skip in your decoder: the linear skip $\mathbf{W}_{\text{dir}}\mathbf{z}$ contributes a constant, $\mathbf{z}$-independent piece to $\mathbf{J}$, so $\mathbf{J}(0)$ is dominated by that fixed map, not by whatever a deep MLP happens to do near zero.

4 Heat: the time-dependent problem

4.1 What we are solving (no ROM yet)

The heat equation with Dirichlet BCs, after FEM discretization in space, gives an ODE in time:

$$\mathbf{M}\,\frac{d\mathbf{u}}{dt} + \mathbf{K}\,\mathbf{u} \;=\; \mathbf{F}, \qquad \mathbf{u}(0) \text{ given.}$$

$\mathbf{M}$ is the mass matrix (also SPD, sparse), $\mathbf{K}$ is the stiffness matrix as before. We discretize time with backward Euler, step size Δt:

$$\mathbf{M}\,\frac{\mathbf{u}_{n+1} - \mathbf{u}_n}{\Delta t} + \mathbf{K}\,\mathbf{u}_{n+1} \;=\; \mathbf{F}.$$

Rearranging:

$$\big(\mathbf{M} + \Delta t\,\mathbf{K}\big)\,\mathbf{u}_{n+1} \;=\; \mathbf{M}\,\mathbf{u}_n + \Delta t\,\mathbf{F}.$$

This is a *linear* system at each timestep. The matrix $\mathbf{M} + \Delta t\,\mathbf{K}$ is SPD (sum of two SPDs). FOM solves this with CG, 50 times in a row.

4.2 Plug in the masked decoder

Replace $\mathbf{u}_{n+1}$ with $\tilde{\mathbf{u}}(\mathbf{z}_{n+1})$ and $\mathbf{u}_n$ with $\tilde{\mathbf{u}}(\mathbf{z}_n)$:

$$\big(\mathbf{M} + \Delta t\,\mathbf{K}\big)\,\tilde{\mathbf{u}}(\mathbf{z}_{n+1}) \;=\; \mathbf{M}\,\tilde{\mathbf{u}}(\mathbf{z}_n) + \Delta t\,\mathbf{F}.$$

Same overdetermined situation as Poisson: N equations, k unknowns. Project with Galerkin — test with $\mathbf{J}(\mathbf{z}_{n+1})$:

$$\mathbf{J}(\mathbf{z}_{n+1})^\top\big(\mathbf{M} + \Delta t\,\mathbf{K}\big)\,\tilde{\mathbf{u}}(\mathbf{z}_{n+1}) \;=\; \mathbf{J}(\mathbf{z}_{n+1})^\top\big(\mathbf{M}\,\tilde{\mathbf{u}}(\mathbf{z}_n) + \Delta t\,\mathbf{F}\big).$$

4.3 The key difference: this is non-linear in $\mathbf{z}_{n+1}$!

Wait. The original full-order step was *linear* in $\mathbf{u}_{n+1}$. But the projected one is non-linear in $\mathbf{z}_{n+1}$, because $\tilde{\mathbf{u}}(\mathbf{z}_{n+1})$ and $\mathbf{J}(\mathbf{z}_{n+1})$ are both non-linear in $\mathbf{z}_{n+1}$.

So we still need an iterative solve. But not a Gauss-Newton outer loop — the simpler *frozen-Jacobian Newton* works because the problem is much closer to linear. The trick: linearize the manifold around $\mathbf{z}_n$ (the previous timestep) and use a single linear solve to step. The textbook NM-ROM linearization gives:

$$\big(\mathbf{J}_n^\top(\mathbf{M} + \Delta t\,\mathbf{K})\,\mathbf{J}_n\big)\,\mathbf{z}_{n+1} \;\approx\; \mathbf{J}_n^\top\big(\mathbf{M}\,\tilde{\mathbf{u}}(\mathbf{z}_n) + \Delta t\,\mathbf{F}\big),$$

where $\mathbf{J}_n = \mathbf{J}(\mathbf{z}_n)$. This is the boxed equation in the Heat subsection of your paper. It is a $k \times k$ SPD linear system, solved with CG. One linear solve per timestep, no outer loop.

Why no Gauss-Newton outer loop here. Two reasons. (i) The full-order step is exactly linear in $\mathbf{u}_{n+1}$, so the only non-linearity in the projected step comes from $\tilde{\mathbf{u}}$ varying along the manifold. (ii) We warm-start: $\mathbf{z}_{n+1} \approx \mathbf{z}_n$ since the solution does not jump much in one timestep. So the Jacobian $\mathbf{J}_n$ is a very good approximation of $\mathbf{J}(\mathbf{z}_{n+1})$, and one frozen-Jacobian solve gets us there.

Compare to Poisson: there is no previous timestep to warm-start from, so we cold-start at $\mathbf{z} = 0$ and need real Gauss-Newton iterations.

4.4 What the Heat code actually does (with EQ)

Your Heat code has one extra wrinkle: it does the projection on *Empirical Quadrature* nodes only, not the full mesh. Look at `nm_rom_heat_vitcp.py` lines 486–513. The structure is:

```
fn  = f_norm(z, kappa)            # forward op on EQ nodes (n_eq,)
R   = s * fn - u_prev_eq          # residual on EQ nodes (n_eq,)
J   = jacfwd(f_norm)(z)           # Jacobian on EQ nodes (n_eq, k)

WJ    = eq_w[:, None] * J         # diag(w) * J   (n_eq, k)
JtWJ  = J.T @ WJ                  # J.T diag(w) J (k, k)
JtWR  = J.T @ (eq_w * R)          # J.T diag(w) R (k,)

# solve: (s^2 JtWJ) dz = -s JtWR        -- with augmentation for s
```

The shape is the same as Poisson: $\mathbf{J}^\top\mathbf{W}\mathbf{J}$ on the LHS, $\mathbf{J}^\top\mathbf{W}\mathbf{R}$ on the RHS, with $\mathbf{W} = \mathrm{diag}(\mathbf{w}_{\mathrm{eq}})$ being the diagonal matrix of EQ weights. This is still Galerkin, just over the EQ subset rather than the full mesh.

Why EQ is required (not optional) for Heat. Without EQ, Heat-3D diverges to rel-$L^2 \approx 1.7\times10^{12}$ — this is in your sweep results. The full-mesh projected operator $\mathbf{J}^\top(\mathbf{M}+\Delta t\,\mathbf{K})\mathbf{J}$ becomes ill-conditioned across the 50-step rollout. EQ implicitly regularizes by restricting to a NNLS-selected subset of well-resolved nodes. For Poisson this is just a speedup; for Heat it is correctness.

5 Empirical Quadrature: where the speedup comes from

The reduced residual in Galerkin is:

$$\mathbf{r}_{\mathrm{red}}(\mathbf{z}) \;=\; \mathbf{J}(\mathbf{z})^\top\big(\mathbf{K}\,\tilde{\mathbf{u}}(\mathbf{z}) - \mathbf{F}\big) \;=\; \sum_{i=1}^{N} \mathbf{J}_{i,:}^\top\,\mathbf{R}_i.$$

That sum is over all N mesh nodes. Even though we project down to k dimensions, we still pay $O(N)$ to compute the sum.

EQ replaces the full sum with a weighted sum over a sparse subset $\mathcal{S} \subset \{1,\dots,N\}$ of size $|\mathcal{S}| = N_{\mathrm{eq}} \ll N$:

$$\mathbf{r}_{\mathrm{red}}(\mathbf{z}) \;\approx\; \sum_{i\in\mathcal{S}} w_i \cdot \mathbf{J}_{i,:}^\top\,\mathbf{R}_i.$$

The weights $\mathbf{w} \geq \mathbf{0}$ are chosen so that this weighted sum reproduces the full sum on the training snapshots. They are found by solving a non-negative least-squares (NNLS) problem on the snapshot data.

Why this is a real speedup. The decoder $\tilde{\mathbf{u}}$ and its Jacobian $\mathbf{J}$ only need to be evaluated at the EQ nodes. With our CP-tensor decoder, evaluating $\tilde{\mathbf{u}}$ at one node costs $O(R\,d)$ (rank times spatial dimension), independent of N_g^d. So the whole reduced residual costs $O(R\,d\,N_{\mathrm{eq}})$ instead of $O(R\,d\,N_g^d)$. Typical ratio in your experiments: $N_{\mathrm{eq}}/N_g^d \approx 0.3\%$.

NNLS in math. Stack the per-node integrands across all training states ℓ into a tall matrix $\mathbf{G}$:

$$\mathbf{G}_{(\ell,i),:} \;=\; \big[\mathbf{J}(\mathbf{z}^{(\ell)})^\top\big]_{i,:}\,\big[\mathbf{K}\,\tilde{\mathbf{u}}(\mathbf{z}^{(\ell)}) - \mathbf{F}\big]_i \;\in\; \mathbb{R}^k.$$

$\mathbf{G} \in \mathbb{R}^{(n_{\text{snap}}\, N)\times k}$. The target is the full sum at each training state:

$$\mathbf{b} \;=\; \sum_{\ell} \mathbf{J}(\mathbf{z}^{(\ell)})^{\top}\, \mathbf{R}^{(\ell)}.$$

Solve

$$\min_{\mathbf{w}\geq \mathbf{0}} \; \|\mathbf{G}^{\top}\mathbf{w} - \mathbf{b}\|_2.$$

NNLS is the algorithm that handles the non-negativity. The output $\mathbf{w}$ is naturally sparse: most w_i come out to zero, and the rest are positive weights. That is what gives you the $|\mathcal{S}| \ll N$ subset.

6 The full picture in one diagram

Inference for one Poisson solve:

```
z = 0                                          # cold start
loop GN iterations (typically 8-15):
  u_pred = D(z)                                # decoder forward pass
  R      = K @ u_pred - F                      # full residual, R^N
  r_red  = J(z).T @ R                          # via vjp; R^k
  loop CG iterations (typically 10-30):
    gn_op(dz) = J(z).T @ K @ J(z) @ dz         # via jvp + K + vjp
  dz = solve gn_op(dz) = -r_red                # k x k system
  z  = z + dz
  break if ||r_red|| small
return D(z)                                    # full-field solution
```

Inference for one Heat solve (50 timesteps):

```
z = encoder(u_0)                               # warm start from IC
for n in 0 .. 49:
  # build Galerkin LHS and RHS using J(z_n) on EQ nodes only
  LHS(dz) = J.T @ W @ (M + dt*K) @ J @ dz      # k x k operator
  RHS     = J.T @ W @ (M @ u_eq_prev + dt*F)   # k vector
  z_next  = solve LHS(z_next) = RHS            # one linear solve
  z = z_next
return D(z)                                    # final state
```

7 Quick reference: what every symbol means

$\mathbf{u} \in \mathbb{R}^N$	full-order PDE state, $N = N_g^d$
$\mathbf{z} \in \mathbb{R}^k$	latent code, k small (≤ 64)
$\mathcal{D}(\mathbf{z})$	decoder, raw output
$\tilde{\mathbf{u}}(\mathbf{z})$	masked decoder, BC-enforced
$\mathbf{m} \in \{0,1\}^N$	Dirichlet mask (0 at boundary)
$\mathbf{u}_g \in \mathbb{R}^N$	Dirichlet lift (boundary values)
$\mathbf{J}(\mathbf{z}) \in \mathbb{R}^{N \times k}$	decoder Jacobian
$\mathbf{K} \in \mathbb{R}^{N \times N}$	FEM stiffness matrix (SPD, sparse)
$\mathbf{M} \in \mathbb{R}^{N \times N}$	FEM mass matrix (SPD, sparse)
$\mathbf{F} \in \mathbb{R}^N$	forcing vector
$\mathbf{R}(\mathbf{z}) \in \mathbb{R}^N$	full residual
$\mathbf{r}_{\text{red}}(\mathbf{z}) \in \mathbb{R}^k$	reduced residual ($\mathbf{J}^\top \mathbf{R}$)
$\mathbf{H}(\mathbf{z}) \in \mathbb{R}^{k \times k}$	reduced Hessian ($\mathbf{J}^\top \mathbf{K} \mathbf{J}$)
$\mathcal{S}$	EQ node subset, $\lvert\mathcal{S}\rvert = N_{\text{eq}} \ll N$
$\mathbf{w}_{\text{eq}} \geq 0$	EQ weights from NNLS
$\mathbf{W} = \text{diag}(\mathbf{w}_{\text{eq}})$	EQ weight matrix

---

End. Read this once, then re-read your code. The mapping is 1-to-1 once you see it.